



Automated Detection and Mitigation of Requirement Bad Smells in Software Requirement Specifications

A. Fathima, K. Indulekha, and Shweta^(✉) 

National Institute of Technology Calicut, Calicut, Kerala, India
{fathima_p250088cs, indulekha_p240086cs, shweta}@nitc.ac.in

Abstract. Software Requirement Specifications (SRS) are typically written in natural language and may contain various requirement bad smells, including imprecise pronouns, missing information indicators, comparative expressions, and non-atomic requirements. The presence of these issues reduces the clarity of SRS documents, limiting their effectiveness for further automation tasks such as automated modelling, testing etc. The existing research primarily focused on a limited number of SRS documents and the identification of a restricted range of bad smells within requirement documents. This paper studies the prevalence of requirement bad smells in industry-level SRS documents and introduces an automated approach for their detection and mitigation. The proposed method employs rule-based linguistic pattern recognition to detect ten different categories of requirement bad smells and uses generative models, i.e., Gemini API to find solutions to fix incorrect requirement statements. The experiment has been conducted on publicly available datasets, i.e., the PURE dataset, which comprises seventy-nine SRS documents with 34,268 sentences. The study identified 25,266 instances of requirement bad smell, comprising 2,147 passive voice constructions, 1,589 conditional modal expressions, 3,638 missing information indicators, 11,310 ambiguous pronoun occurrences, 749 comparative terms, 3,903 non-atomic requirements, 1,115 undefined abbreviations, and 815 subjective expressions. The results show that bad smells are common in real-world specifications. Automatic detection and mitigation of bad smells could make the requirements document clearer and easier to comprehend.

Keywords: Requirements Engineering · Software Requirements Specification · Requirement Bad Smells

1 Introduction

Software Requirement Specifications (SRS) document plays an important role in the software development life cycle. The document involves a detailed description of software requirements, and the high quality of requirements is the medium of connection between software development team and the customer.

The SRS document usually written in natural language text, i.e. English and hence, it can contain ambiguity and inconsistency. The document contains several repeating issues such as use of conditional modal verbs ‘*could*’, ‘*might*’, or ‘*should*’, which weaken the enforceability of a requirement, vague pronouns like ‘*it*’ or ‘*this*’ can create ambiguity when the referent is not clearly specified, and missing information, i.e. ‘TBD’, etc. These types of linguistic patterns are known as ‘requirement bad smells’ that make the SRS document ambiguous and harder to understand. The existing research primarily focused on a limited number of SRS documents and the identification of a restricted range of bad smells within requirement documents. Therefore, this work aims to identify a wider spectrum of these bad smells and provide practical suggestions for their mitigation and strives to determine the extent to which bad smells are present in the existing publicly available software requirement specification documents. Hence, the presented automated framework detects requirement bad smells and also provides suggestions to mitigate the bad smells. The methodology takes SRS documents as input and identifies the problematic patterns, i.e. requirement bad smells, and uses a generative model to provide suggestions for the removal of bad smells.

The organization of the rest of the paper is as follows: Sect. 2 contains a review of the field, which includes past research done on requirement bad smell detection and requirement quality analysis. Section 3 describes the proposed framework for automated detection and removal of requirement bad smells. Section 4 presents the experimental analysis discusses the findings, limitations, and possible directions for future work. Finally, Sect. 5 concludes the paper.

2 Related Work

Several studies have investigated techniques for detecting and improving requirement quality. Here, a few of them have been summarised as follows:

Ferrari et al. in [5] introduced the PURE dataset, which includes publicly available SRS documents for requirements engineering research. The dataset includes 34,268 sentences and can be used for natural language processing tasks that are typical in requirements engineering. The dataset contains a preliminary version of XML schema and textual files for each SRS.

In a study [4], authors introduced the concept of requirement smells and developed automated tools like Smella to identify bad smell patterns in requirement statements. The empirical evaluation involved industrial contexts from various companies such as Daimler AG, Wacker Chemie, and TechDivision GmbH. The Smella have been employed to 51 requirements artifacts created by students. The different types of bad smells are identified in the paper are Subjective language, Ambiguous adverbs and adjectives, Loopholes, Open-ended / non-verifiable terms, Comparative terms, Superlative terms, Negative statements and Vague pronouns. However, the experience of the students as well as the practitioners might play a role in their ratings of relevance or detection of quality defects.

In the work by Pandiyavathi and Sivakumar [7] proposed an ensemble deep learning approach for the prediction of code smells in software projects. The authors focused on source code-level smell prediction using a dataset containing software projects collected from different online repositories. These projects are labelled based on Security level, refactoring needs, and code smell presence. These are used to train an ensemble network. This paper mainly focused on bad smells such as Code-level bad smells affecting software quality and security, Design inconsistencies, Maintainability issues, and quality attribute conflicts. However, this method mainly focuses on source code analysis for predicting the presence of code smells.

In the work by Fantechi et al. [3], which employed large language models in identifying inconsistency defects in requirements. The paper aims at only two types of bad smells, i.e., requirement inconsistency and conflicting functional descriptions. They added artificial, inconsistent requirements to create a mutated dataset. However, the authors experimented with only four requirement documents from the Coffee machine system, the E-shop system, the Library administration system and the European Train Control System, which makes the proposed method less scalable to other domains.

Dutra et al. [1] conducted a systematic survey on the topics of microservice architectural bad smells detection, and refactoring. The study uses 27 primary research papers collected from six digital libraries, a compiled catalogue of 104 microservice bad smells, and the identification of 7 smell detection tools. The resulting structured dataset became a reference for future empirical studies. The survey aims at 12 architectural categories of bad smells, such as API management smells, dependency management smells, data management smells, devOps / CI-CD smells, decomposition smells, migration smells, cyclic dependency, hard-coded endpoints, and hub-like dependency.

In another study [2], authors presented a deep learning approach to the task of identifying anaphoric ambiguity in requirements, which improves requirement clarity by identifying vague pronoun reference using a curated dataset DAMIR (standing for dataset for Anaphoric Ambiguity In Requirements). The dataset contains requirements with annotated ambiguous pronoun references. This paper only focuses on anaphoric ambiguity bad smells, and the system cannot handle any other type of requirement bad smells.

Finally, Veizaga et al. [8] presented an automated framework for requirement bad smells detection and improvement recommendations. They proposed an automated NLP-based approach named PASKA for detecting requirement bad smells. Their methodological approach also gives recommendations for improvements using a language called Rimay. The paper uses an industrial case study dataset using 13 software requirement specifications. The dataset contained 2725 manually annotated natural language requirements. The approach detects nine types of requirement smells, such as missing requirement segments, incorrect ordering of requirement components, ambiguous or unclear requirements, lack of atomicity, completeness violations, syntax and semantic structure errors. The system implements various natural language processing approaches. The results

are susceptible to annotation biases as they have experimented with a small number of requirements documents.

The existing work mainly experimented on a small number of SRS documents and has identified a limited number of types of bad smells from requirement documents. Hence, the proposed work strives to identify different types of code smells and provide suggestions to improve the requirement bad smells.

3 Proposed Methodology

The proposed methodology comprises two main tasks: (1) requirement bad smells Detection using linguistic pattern identification, (2) Mitigation of requirement bad smells using a generative model. The following subsection illustrates the methodology step by step (Fig. 1):

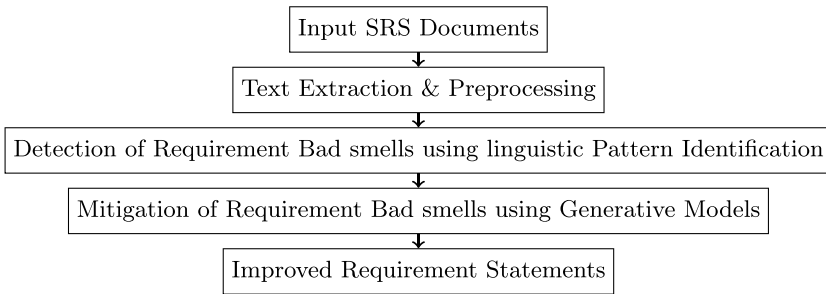


Fig. 1. Proposed Methodology for Automated Detection and Mitigation of Requirement Bad Smells

3.1 Input SRS Documents

The methodology uses the PURE [5] dataset that contains 79 SRS documents with 34,268 requirements sentences. The Software Requirement Specifications include the requirements needed for the successful completion of the development of the software. This dataset contains the SRS document in XML and text format. Further, parsing methods can be used to read XML files. The input is given in the text format.

3.2 Text Extraction and Preprocessing

Text data is parsed and processed using various XML parsing techniques. After that, the text goes through the preparation stages, like splitting up phrases, tokenisation, and stopword removal. Here, the proposed system utilises NLP tools like spaCy to automate these tasks.

3.3 Detection of Requirement Bad Smells Using Linguistic Pattern Identification

The individual sentences are processed using syntactic analysis techniques. Then the sentences are checked for the requirement bad smell using rule-based pattern matching. The following types of bad smells have been detected:

Passive Voice and Conditional Modal Detection: The system uses dependency parsing to detect passive voice, while modal words such as ‘*could*’, ‘*might*’, and ‘*should*’ are detected using part-of-speech tagging. Passive voice detection uses dependency parsing tag, i.e. ‘*nsub:jpass*’ and *auxiliary verbs*.

Conditional Modals are used to define conditional scenarios. Modal verbs are detected using ‘*MD*’ from POS tags. These represent weak and ambiguous requirement statements [4].

Missing Information and Vague Pronoun Detection: The missing information, i.e. ‘*TBD*’, ‘*TBA*’, is detected by keyword matching. Detection of ‘*TBD*’, ‘*TBA*’, and to be defined uses rule-based filtering techniques.

For vague pronouns, basic anaphora resolution is used. Vague pronouns use simple heuristics based on co-reference detection. These represent missing and ambiguous requirements. [2, 8].

Comparative and Non-Atomic Requirement Detection: Comparatives are detected by using ‘*JJ*’ and ‘*JJS*’ tags. Comparative and superlative adjectives use ‘*JJR*’ and ‘*JJS*’ from POS tags.

Non-atomic requirements are detected by the use of conjunctions such as ‘*and*’, ‘*or*’. Detection of multiple actions in a single sentence uses conjunction-based splitting techniques. These reduce requirement precision and testability [4].

Sentence Length and Complexity Detection: The complexity of sentences is determined by the number of words, the number of clauses, and the syntactic parsing trees. Sentence length are determined by filtering based on thresholds based on flagging the sentences that exceeds a certain word count. The proposed system counts tokens per sentence and flags those shorter than 3 or longer than 40 words.

Sentence complexity uses token count and clause detection from dependency parsing techniques. Detection of long sentences uses threshold-based filtering techniques. Sentence complexity negatively impacts sentence readability and clarity [8].

Abbreviation and Subjective Term Detection: The detection of abbreviations is based on regex pattern matching. Detection of abbreviations uses regex pattern matching techniques. Regex pattern matching are used to match, validate, extract or modify strings in the sentences.

The detection of subjective terms is based on predefined dictionaries. Detection of subjective terms uses a predefined list of subjective terms. These negatively impact requirement ambiguity and measurability [4, 8].

3.4 Mitigation of Requirement Bad Smells Using Generative Models:

The sentences with problems are given to Gemini-2.5 Flash [6]. The Gemini API processes the sentences and produces the suggestions for corrected versions of the requirements.

Improved Requirement Statements. The output will include the suggestions for requirement sentences with improvements and corrections. The requirement bad smells identified will be improved by using the Gemini API for artificial intelligence-based rewriting [3,8].

Passive Voice and Conditional Modal Resolution: The passive voice is changed to an active voice. Sentences in passive voice are rewritten in active voice with explicit agents added.

- *Passive Voice Detected:* “The data shall be validated before processing.”
- **Resolved:** “The system shall validate the data before processing.”

The weak modals are replaced with strong requirement words like ‘shall’. Weak modal verbs are replaced with ‘shall’ or ‘must’. This makes the sentences more explicit, clear, and easier to enforce [3].

- **Conditional Modal Detected:** “The module might fail”.
- **Resolved:** “The module shall fail”.

Missing Information and Vague Pronoun Resolution: Meaningful placeholders are used in the replacement, while unclear pronouns are resolved. Placeholders are replaced with domain-specific values, if available.

- **Missing Information Detected:** “The part of this process that will issue the signal is designated as TBD(to be determined)”.
- **Resolved:** “Replace placeholder values with concrete specification details”.

Ambiguous pronouns are replaced with explicit ones. This makes the sentences complete, clear, and easier to understand [2,3].

- **Vague Pronoun Detected:** “This module processes data.”
- **Resolved:** “The data processing module processes data.”

DashComparative and Non-Atomic Resolution: Comparatives are replaced with quantifiable metrics. Sentences with comparative terms are replaced with quantifiable values like response time or limits.

- **Comparative Detected:** “Short pages are generally more appropriate for homepages, navigation on pages, or overview pages that need to be read quickly.”
- **Resolved:** “Homepages, navigation on pages, and overview pages shall limit textual content to a maximum of 150 words to facilitate quick comprehension.”

Non-atomic sentences are broken down into separate sentences. This makes the sentences verifiable [8].

- **Non Atomic Requirements Detected:** *“The system logs events and notifies users.”*
- **Resolved:** *“The system logs events.” “The system notifies users.”*

Sentence Length and Complexity Analysis: Sentence length is identified by measuring mean number of words per sentence. Most SRS sentences ranged between 10–30 tokens, with some exceeding 40 due to compound structures.

Complex sentences are rewritten in a simpler form using AI technology. Complex sentences are broken down into shorter sentences using AI-based rewriting tools. Restructuring the sentences makes them easier to understand [3].

- **Sentence Length and Complexity Detected:** *“Making text resizable by the user: Text should be able to be resized by the user, using functions provided by the user agent or other appropriate means”.*
- **Resolved:** *“Users must be able to resize text. They can use user agent functions. Other suitable methods are also fine”*

Abbreviation and Subjective Term Resolution: Abbreviations are resolved on the first occurrence. Abbreviations are replaced with the full form on the first occurrence of the abbreviation.

- **Abbreviation Term Detected:** *“The SRS shall be reviewed by QA.”*
- **Resolved:** *“Provide abbreviation expansion at first occurrence (e.g., Quality Assurance (QA)).”*

Subjective terms are replaced with quantifiable criteria. Subjective terms are replaced with quantifiable values [4,8].

- **Subjective Term Detected:** *“The system shall be fast.”*
- **Resolved:** *“The system shall respond within 2 s.”*

4 Results and Discussions

The bad smell detection results have been compared with the manual identification method, as shown in Table 1. Two SRS files, i.e., gamma j.xml [5](SRS 1) and eirene_fun_7-2.xml [5] (SRS 2), have been used from the PURE dataset for evaluation of the proposed method. The results indicate that the proposed methodology identifies slightly more bad smells than the manual method, suggesting a reduced risk of false negatives. But in some cases like Non atomic requirements, Missing information and Passive voice type of requirement bad smell, both the methods seems consistent. So from this comparison it is clear that the proposed methodology will effectively identify all the mentioned type of requirement bad smells.

Table 1. Frequency Distribution of Requirement Bad Smells

Requirement Bad Smell Type	Frequency	SRS 1 [5]		SRS 2 [5]	
		Proposed	Manual	Proposed	Manual
Ambiguous Pronouns	11,310	44	30	200	181
Passive Voice	8,067	58	51	405	358
Non-Atomic Requirements	3,903	132	132	310	310
Missing Information	3,638	0	0	0	0
Conditional Modals	1,995	18	5	161	73
Abbreviation Issues	1,115	0	6	0	11
Complex and Long Sentences	1,820	33	23	445	311
Subjective Terms	815	8	8	2	2
Comparative Terms	749	10	9	19	12
Superlative Terms	371	2	3	14	7
Total Bad Smell Instances	34,163	-	-	-	-

The proposed methodology detected different bad smells from the PURE dataset and found significant variation among all the different types of requirement bad smells, as shown in Table 1.

Among all the detected requirement bad smells, ambiguous pronouns are identified as the most frequent requirement bad smell, which is 11310 from the total of 34,268 sentences. This shows that unclear references to ‘it’, ‘this’, and ‘they’ etc. like pronouns are a major source of this type of requirement bad smell. The least frequent requirement bad smell is superlative terms with 371 occurrences. These frequency counts are more effectively visualized using a bar graph, as presented in Fig. 2. It indicates that superlative expressions such as ‘best performance’, ‘most beautiful’ etc. are very rare in the documents.

The presented automated framework identifies different bad smells and provides suggestions to improve the quality of requirement documents. The rule-based linguistic analyzer, augmented with a generative model for detection and mitigation suggestions, helps make requirements precise and easier to validate. Hence, improving record-keeping in large-scale software projects. However, the presented work has some major limitations as well: 1) The framework has been evaluated solely on the PURE dataset. To obtain broader insights across different SRS domains, it should be further validated on other industry-relevant requirements documents. 2) The study only presented preliminary results and lacks human validation, which may impact the practical usability and correctness. 3) The system does not assess the quality of the suggestions generated by the model; therefore, further work is needed to evaluate these automatically generated suggestions.

In future works, this research will look at including the integration of the framework with requirement management tools. This will give instant feedback during the development of the requirements. It can be extended by using deep

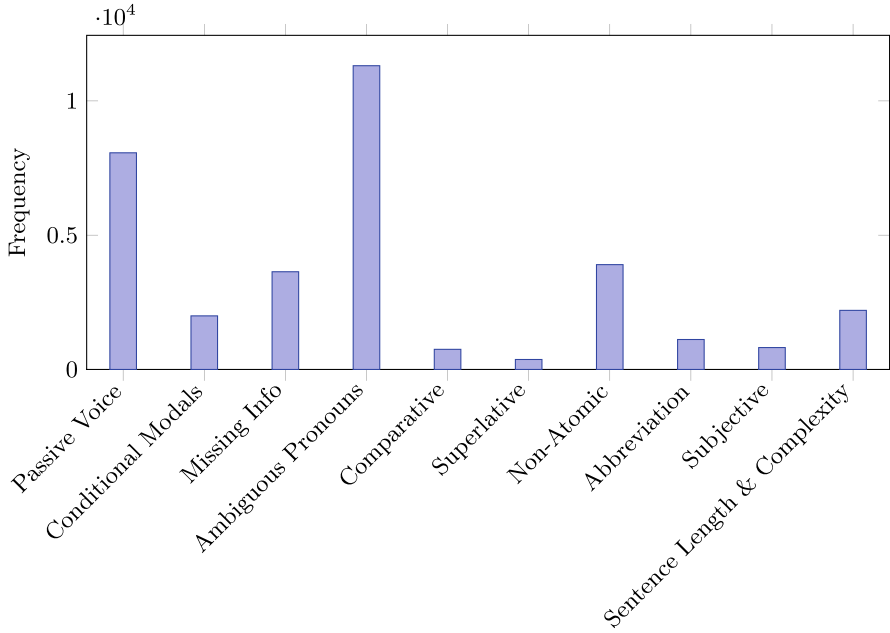


Fig. 2. Distribution of Requirement Bad Smells

learning models for enhancing the framework for adapting different types of SRS documents, as well as adding more evaluation metrics to improve the quality of suggestions.

5 Conclusion

The paper presents a study that investigates the extent to which industry-level SRS documents exhibit requirement bad smells and proposes an automated approach for their detection and mitigation. The proposed approach employed linguistic patterns to detect requirement bad smells and identify a wide range of such issues in significant numbers. A generative model has been used to propose suggestions to reduce the detected bad smells. The results show that the automatic detection and mitigation of bad smells indeed make the requirements clearer and easier to work with. This presented framework for the detection and mitigation of bad smells can be integrated with requirement management tools to automatically get suggestions while documenting software requirements.

References

1. Dutra, M., Pinheiro, D., Oliveira, J., Figueiredo, E.: From detection to refactoring of microservice bad smells: a systematic literature review. *J. Softw. Eng. Res. Dev.* **13**(2), 172–183 (2025). <https://doi.org/10.5753/jserd.2025.5224>

2. Ezzini, S., Abualhaija, S., Arora, C., Sabetzadeh, M.: Taphsir: Towards anaphoric ambiguity detection and resolution in requirements. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022), pp. 1677–1681. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3540250.3558928>
3. Fantechi, A., Gnesi, S., Passaro, L., Semini, L.: Inconsistency detection in natural language requirements using ChatGPT: a preliminary evaluation. In: Proceedings of the 2023 IEEE 31st International Requirements Engineering Conference (RE), pp. 335–340 (2023). <https://doi.org/10.1109/RE57278.2023.00045>
4. Femmer, H., Méndez Fernández, D., Wagner, S., Eder, S.: Rapid quality assurance with requirements smells. *J. Syst. Softw.* **123**, 190–213 (2017). <https://doi.org/10.1016/j.jss.2016.02.047>
5. Ferrari, A., Spagnolo, G.O., Gnesi, S.: Pure: a dataset of public requirements documents (2018). <https://doi.org/10.5281/zenodo.1414117>
6. Gemini Team, Google: Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities (2025). arXiv preprint [arXiv:2507.06261](https://arxiv.org/abs/2507.06261)
7. Pandiyavathi, T., Sivakumar, B.: Ensemble deep network for secured refactoring framework by predicting code-bad smells in software projects. *J. Softw. Evol. Process* **37**, e2749 (2025). <https://doi.org/10.1002/smr.2749>
8. Veizaga, A., Shin, S.Y., Briand, L.C.: Automated smell detection and recommendation in natural language requirements. *IEEE Trans. Software Eng.* **50**(4), 695–720 (2024). <https://doi.org/10.1109/TSE.2024.3361033>